MAIE 5532: Machine Learning Systems

Week 2: Automatic Differentiation

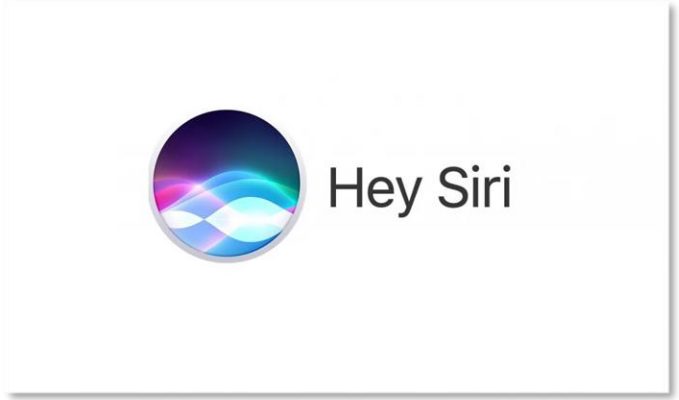
Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lefteris NTAFLOS, PhD

The Hong Kong University of Science and Technology

Week 2 Overview



Automatic Differentiation: The Engine of Modern ML

Learning Objectives:

- Master forward and reverse mode automatic differentiation algorithms
- Analyze memory and computational trade-offs between AD modes
- Evaluate AD implementation strategies for resource-constrained environments
- Design efficient gradient computation systems for edge and embedded deployment

TensorFlow

PyTorch

Required Readings:

- **Book 1 - Volume 1:** Neural Computation
- **Book 1 - Volume 1:** ML Frameworks
- **Book 1 - Volume 1:** Model Training
- **Book 2:** Chapter 3: Getting Up to Speed on Machine Learning

The Fundamental Challenge

Why Automatic Differentiation Matters

The Problem:

Machine learning frameworks must solve a fundamental computational challenge: calculating derivatives through complex chains of mathematical operations efficiently and accurately.

Consider this complex function: $f(x) = x^2 \sin(x) e^{\cos(x)}$

~~Traditional Approach: Manual Calculus~~

175 billion parameters

ML model parameters are internal variables, such as weights and biases, that a model learns from training data to make predictions.

Even in this basic example, computing derivatives manually would require careful application of calculus rules. Now imagine scaling this to a neural network with millions of operations.

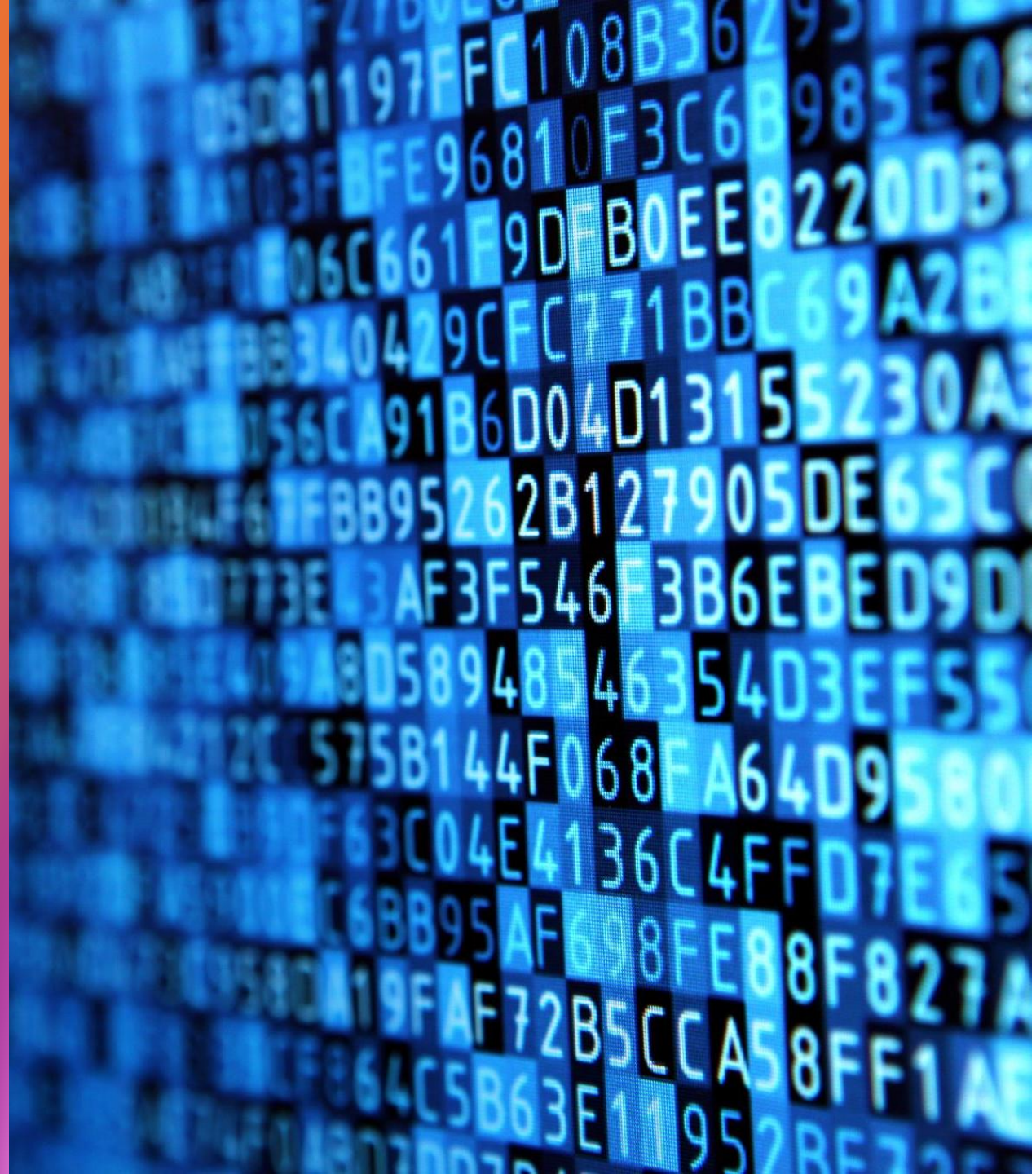
The Breakthrough Insight

Car dashboard analogy: Speedometer + Odometer

Traditional approach: checking odometer for position, speedometer for speed

Automatic differentiation: a futuristic dashboard that gives you both simultaneously

Position & Speed ⟺ Value & Derivative



Dual Numbers - The Secret Weapon

Dual(value, derivative)

$$x = \boldsymbol{Dual}(\mathbf{2.0}, \mathbf{1.0})$$

Regular number:

'I'm at position 5.'

Dual number

'I'm at position 5, moving at speed 2.'

Session Structure

Today's Organization

Session 1: Forward Mode AD
- Forward mode principles and implementation
- Dual number representation
- Performance characteristics and use cases

Session 2: Reverse Mode AD
- Reverse mode algorithms and memory management
- Backward propagation mechanics
- System-level optimization strategies

Session 3: Embedded Systems and Practical Applications
- AD in resource-constrained environments
- TinyML considerations and trade-offs
- Hands-on implementation exercise

Forward Mode AD

Computing Derivatives Alongside Values

Core Concept:

Forward mode automatic differentiation computes derivatives alongside the original computation, tracking how changes propagate from input to output.

TinyML Context: Real-World Constraints

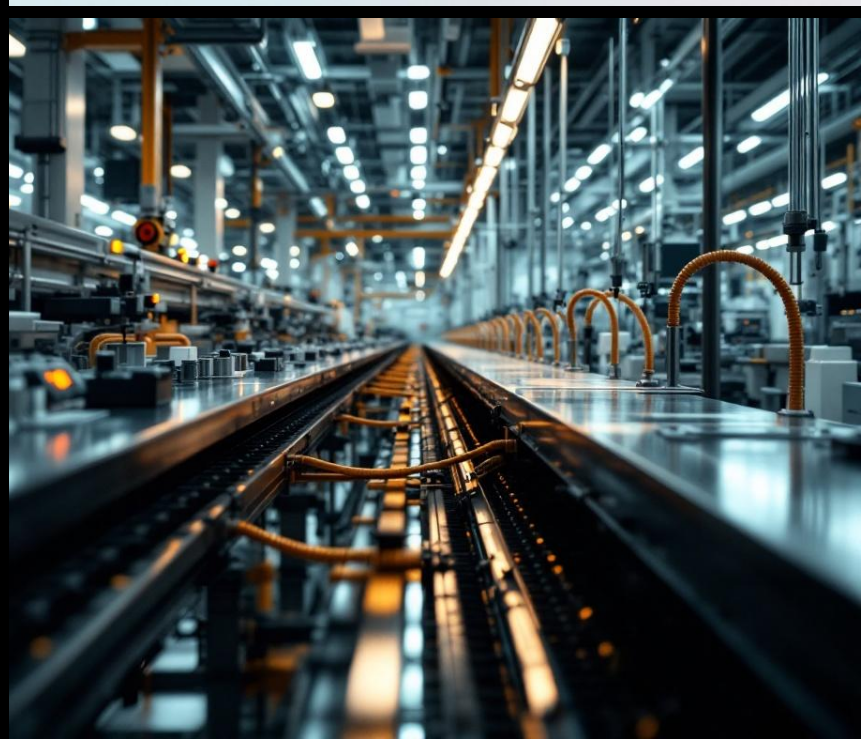
1) Wake Word Detection Example:

- Process 1-second audio windows (~16,000 samples)
- Network with 10,000-50,000 parameters
- Memory constraint: < 32KB total RAM
- Energy constraint: < 1mW power consumption
- The Challenge: Computing gradients in resource-constrained environments

2) Industrial IoT (Factory Prediction)

```
def factory_prediction(temperature, vibration, production_rate):
    normalized_temp = (temperature - 30) / 50  # Normalization step
    vibration_factor = vibration * 0.8
    combined = normalized_temp * vibration_factor + production_rate
    risk_score = 1 / (1 + exp(-combined))  # Sigmoid activation
    return risk_score
```

Question: How do we compute $\partial(risk\_score)/\partial(temperature)$ efficiently?



The Magic of Automatic Calculus

Dual Number Multiplication Rule

$(a, b) \times (c, d) = (a \times c, a \times d + b \times c)$

Calculus Product Rule

$(f * g)' = f' * g + f * g'$

Arithmetic Operations Are Calculus Rules

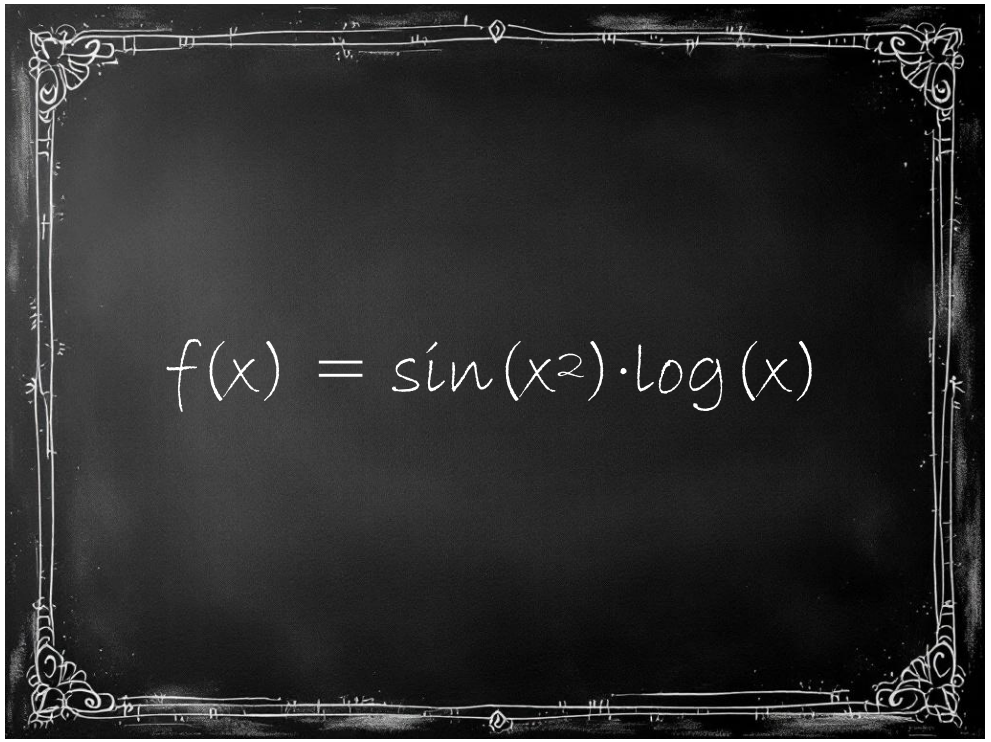
- Addition: $(a,b) + (c,d) = (a+c, b+d)$ → The Sum Rule: $(f+g)' = f' + g'$
- Multiplication: $(a,b) \times (c,d) = (a \times c, a \times d + b \times c)$ → The Product Rule: $(f \times g)' = f' \times g + f \times g'$
- The Result: By doing simple arithmetic, we automatically compute exact derivatives.

The Core Idea: Dual Numbers Encode Calculus via a simple "evaluation process":

- **Algebraic form of Dual number**: $a + b\varepsilon$ (ε a special symbol with $\varepsilon^2 = 0$)
- **Convenient ordered-pair form of Dual number: (a, b)**
- **a: the function value, b: the derivative**
- Arithmetic rules mirror calculus rules
- Product rule appears automatically

Why Manual Differentiation Is Tedious

Identify rules → Apply chain/product → Simplify → Evaluate



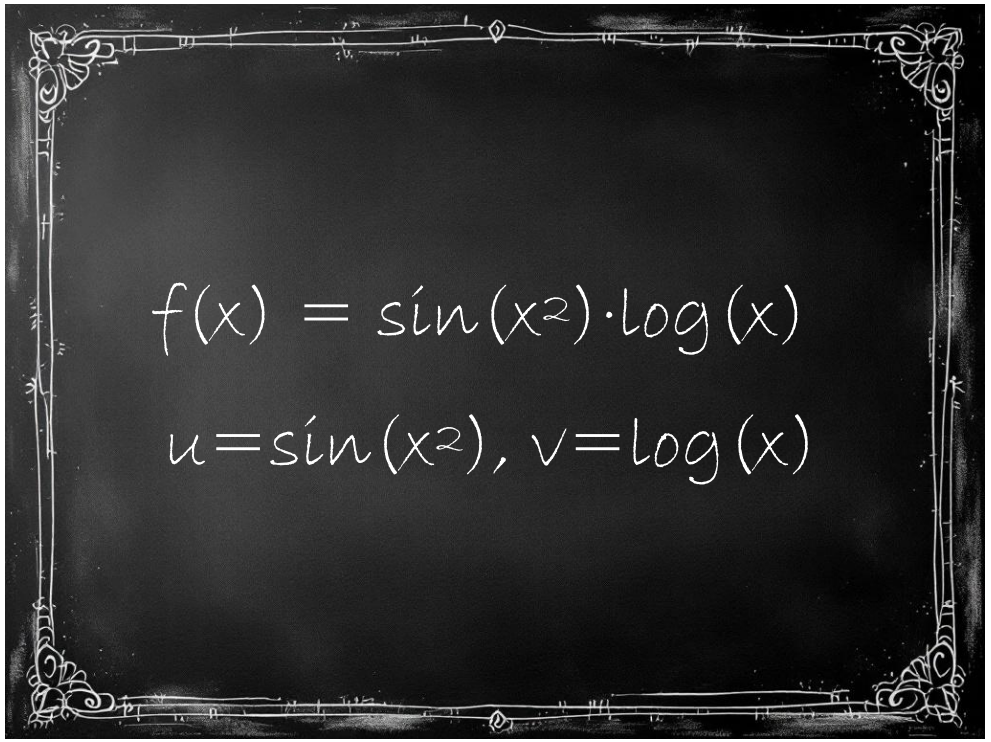
Manual steps:

- Choose rules (product + chain twice)
- Compute u’ and v’
- Apply f’ = u’v + uv’
- Simplify, then plug in x

Time and error prone

Why Manual Differentiation Is Tedious

Identify rules → Apply chain/product → Simplify → Evaluate



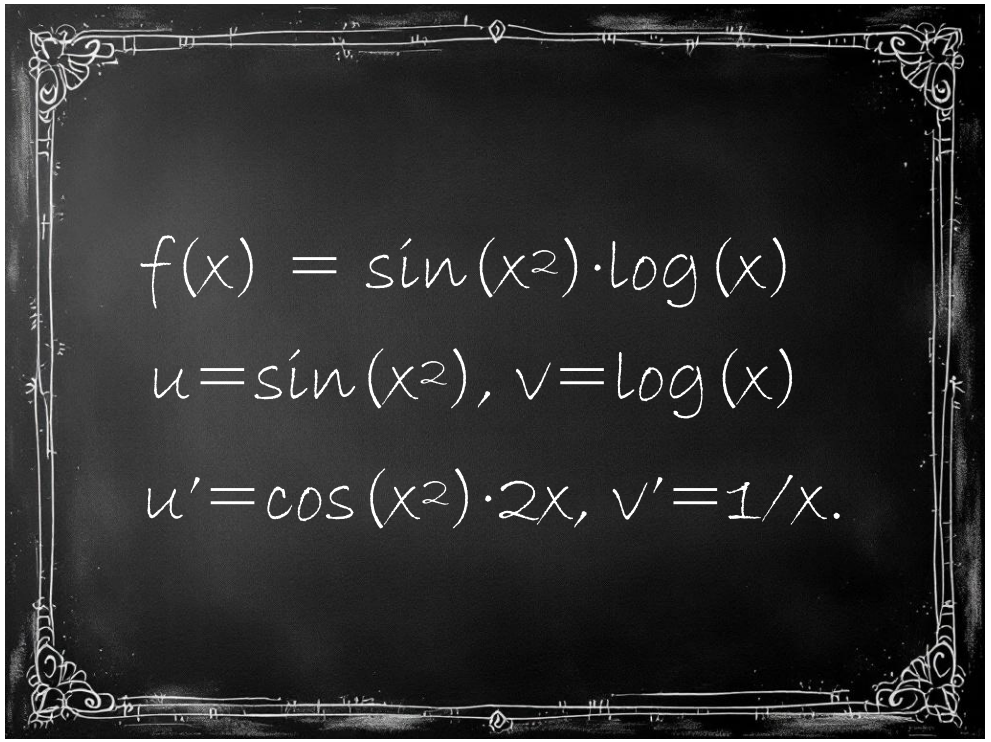
Manual steps:

- Choose rules (product + chain twice)
- Compute u’ and v’
- Apply f’ = u’v + uv’
- Simplify, then plug in x

Time and error prone

Why Manual Differentiation Is Tedious

Identify rules → Apply chain/product → Simplify → Evaluate



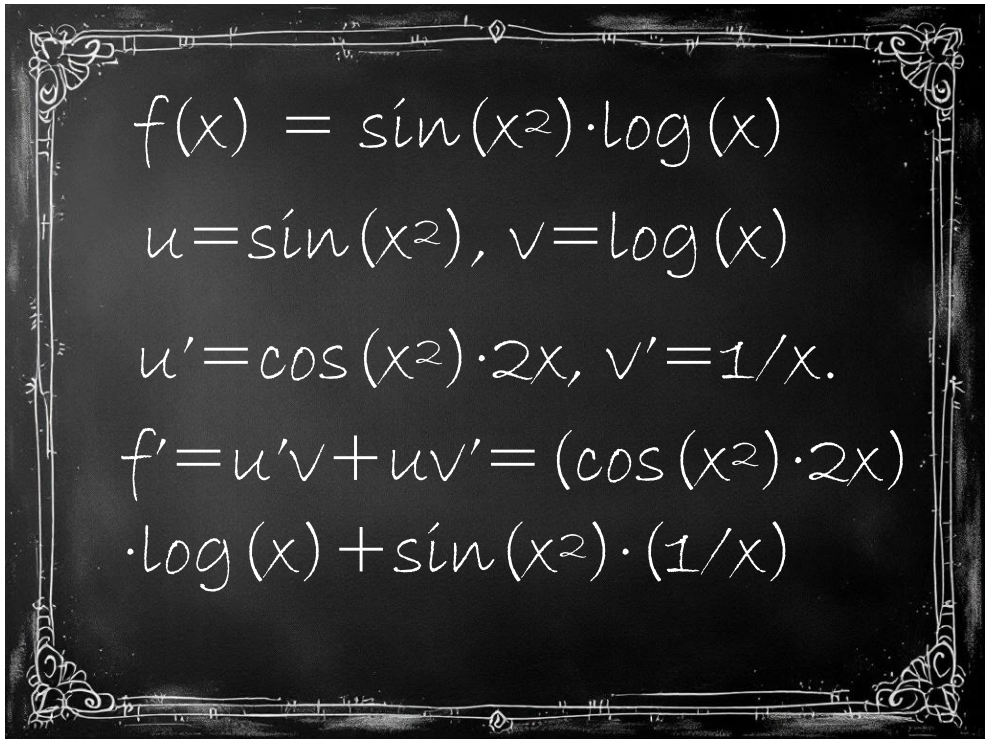
Manual steps:

- Choose rules (product + chain twice)
- Compute u’ and v’
- Apply f’ = u’v + uv’
- Simplify, then plug in x

Time and error prone

Why Manual Differentiation Is Tedious

Identify rules → Apply chain/product → Simplify → Evaluate

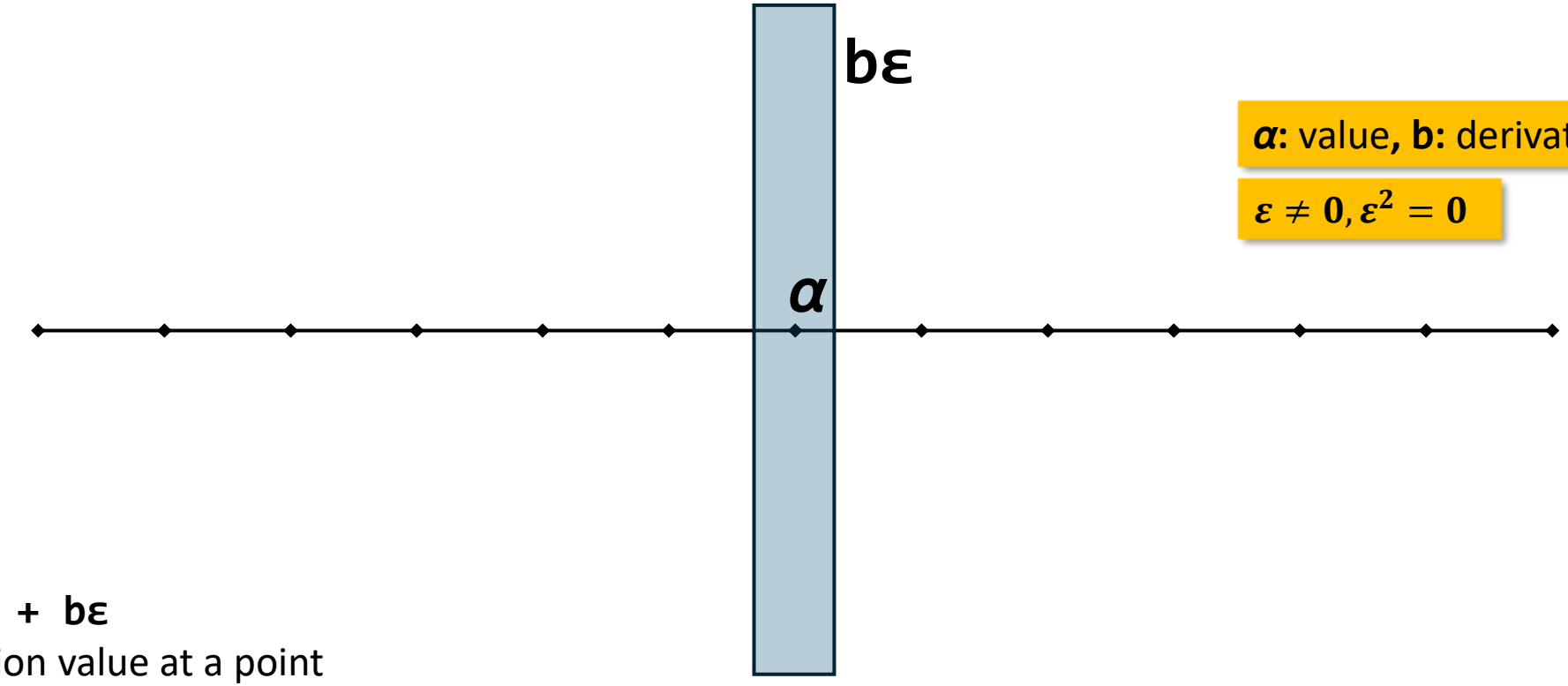


Manual steps:

- Choose rules (product + chain twice)
- Compute u’ and v’
- Apply f’ = u’v + uv’
- Simplify, then plug in x

Time and error prone

What Are Dual Numbers?



***α*: value, b: derivative**

$\varepsilon \neq 0, \varepsilon^2 = 0$

- Form: ***α*** **+ bε**
- ***α***: function value at a point
- **b**: derivative at that point
- Defining property: **$\varepsilon^2 = 0$** (nilpotent)

***Nilpotent**: a matrix or an algebraic element that, when multiplied by itself a sufficient number of times, results in a zero matrix or zero element, respectively*

ε in Action: First-Order Taylor Expansion

The **Taylor series** of a function is an infinite sum of terms calculated from the values of its derivatives at a single point. It's an essential method for **function approximation**, numerical analysis, and solving differential equations.

Standard Taylor Expansion expansion around point a

(**h** is the increment from α to **x**, i.e. $\mathbf{h = x - \alpha}$)

$$f(\alpha + h) = f(\alpha) + f'(\alpha)\cdot h + f''(\alpha)\cdot h^2/2! + f'''(\alpha)\cdot h^3/3! + \ldots$$

ε in Action: First-Order Taylor Expansion

The **Taylor series** of a function is an infinite sum of terms calculated from the values of its derivatives at a single point. It's an essential method for **function approximation**, numerical analysis, and solving differential equations.

Standard Taylor Expansion expansion around point a

(**h** is the increment from **a** to **x**, i.e. $\mathbf{h = x - \alpha}$)

$$f(\alpha + h) = f(\alpha) + f'(\alpha)\cdot h + f''(\alpha)\cdot h^2/2! + f'''(\alpha)\cdot h^3/3! + \ldots$$

$h = b\varepsilon$

$$f(\alpha + b\varepsilon) = f(\alpha) + f'(\alpha)\cdot b\varepsilon$$

Automatic first-order expansion!

Arithmetic on dual numbers mirrors first-order Taylor expansions, giving us the derivative coefficient automatically

*The **first-order Taylor expansion**, or **linear approximation**, approximates a function f(x) around a point a using its value and its first derivative at that point*

Example

Key identity

$f(a+b\varepsilon) = f(a) + f'(a)\cdot b\varepsilon$

Say we have the function $f(x) = x^2$

Step 1: Compute $f(a+ b\varepsilon)$:

$$f(a + b\varepsilon) = (a + b\varepsilon)^2$$

$$= a^2 + 2a(b\varepsilon) + (b\varepsilon)^2$$

$$= a^2 + 2ab\varepsilon + b^2\varepsilon^2$$

$$\mathbf{= a^2 + 2ab\varepsilon}$$

Step 2: Match the pattern $f(a) + f'(a)\cdot b\varepsilon$:

$$a^2 + 2ab\varepsilon = f(a) + f'(a)\cdot b\varepsilon$$

Step 3: Extract the values

$f(a) = a^2$ ✓

$f'(a)\cdot b\varepsilon = 2ab\varepsilon$

Therefore: $f'(a) = 2a$ ✓

Verification: The derivative of x^2 is indeed 2x, so $f'(a) = 2a$ ✓

Arithmetic Rules Recap (And Why They're Calculus)

Dual arithmetic	Calculus rule
$(a,b)+(c,d) \rightarrow (a+c, b+d)$	$(f+g)' = f' + g'$
$(a,b)\times(c,d) \rightarrow (ac, ad+bc)$	$(fg)' = f'g + fg'$

Scalar
$s \times (a,b) = (sa, s \cdot b)$

- Addition = sum rule
- Multiplication = product rule
- Scalars carry derivatives linearly

Worked Example (Manual vs Automatic)

Function: $f(x) = x^2 + 3x$
Input: $x = 2$

By Hand	Dual Numbers
$f(x) = x^2 + 3x$	$x = (2,1)$ (because $dx/dx = 1$)
$f'(x) = 2x + 3$	$x^2 = (2,1) \times (2,1)$ $= (4,2 + 2) = (4,4)$
$f(2) = 10$	$3x = (6,3)$
$f'(2) = 7$	$x^2 + 3x = (4,4) + (6,3)$ $= (10,7)$

A Nonlinear Example: How ε Works

Function: $f(x) = \sin(x^2)\cdot\log(x)$
Input: $x = (2, 1)$

Step 1	$x^2 = (2,1)\times(2,1) = (4,4)$
Step 2	$sin(x^2)$:$sin(4+4\varepsilon)$ $=sin(4)+cos(4)\cdot 4\varepsilon$
Step 3	$log(x)$:$log(2+1\varepsilon)$ $=log(2)+\left(\frac{1}{2}\right)\cdot 1\varepsilon$
Step 4	Multiply: • Primary: $\sin(4)\cdot\log(2)$ • Dual: $\left(\frac{1}{2}\right)\cdot\sin(4)$ $+4\cdot\cos(4)\cdot\log(2)$

Explaining ε with a Concrete Mini-Example

$(3 + 2\varepsilon) \times (5 + 7\varepsilon)$

Expand and cancel ε^2 terms

Expand: $15 + 21\varepsilon + 10\varepsilon + 14\varepsilon^2$
Use: $\varepsilon^2 = 0 \rightarrow 14\varepsilon^2$ drops out
Result: $15 + 31\varepsilon$

Scalar and Constant Rules (Quick Check)

Constant c as (c, 0)
Variable x as (x, 1)
Scalar s times (a, b) → (sa, s·b)

- Constants carry zero derivative: c → (c, 0)
- Independent variable seeds derivative 1: x → (x, 1)
- Linear scaling applies to both parts

Multiple Variables (Preview)

Vector dual numbers:
`(a, b1ε1 + b2ε2)`

Labels: `εiεj = 0` for all `i,j; εiεj = 0` even when `i≠j` in the simplest ‘forward-mode’ model

Example:
`f(x, y) at (x, y) = ((a, 1, 0), (c, 0, 1))`

- Extend to `ε1, ε2, …` with all `εiεj = 0`
- Seed `x` as `(x, 1, 0, …)`, `y` as `(y, 0, 1, …)`
- Result: one pass yields partial derivatives

How You'd Do It Manually vs With ε

Manual: pick rules, chain carefully, distribute, simplify	Dual: plug $x=(a,1)$, evaluate arithmetic, read off coefficient of ε

- Manual:
  - Identify rules per operation
  - Track algebra, avoid mistakes
  - Simplify expressions
- With dual numbers:
  - Replace x by $(a,1)$
  - Evaluate f using dual arithmetic
  - Read derivative from ε coefficient

What ε is (and isn't)

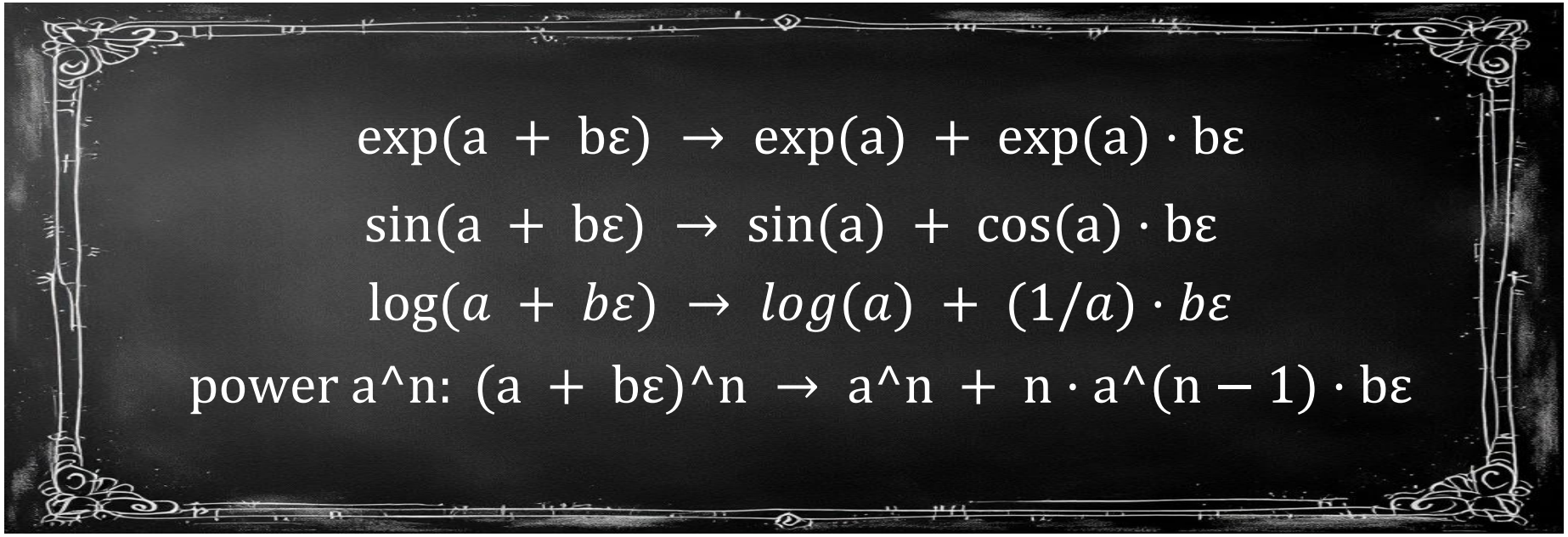
✓ **Nilpotent**

✗ **Not infinitesimal**

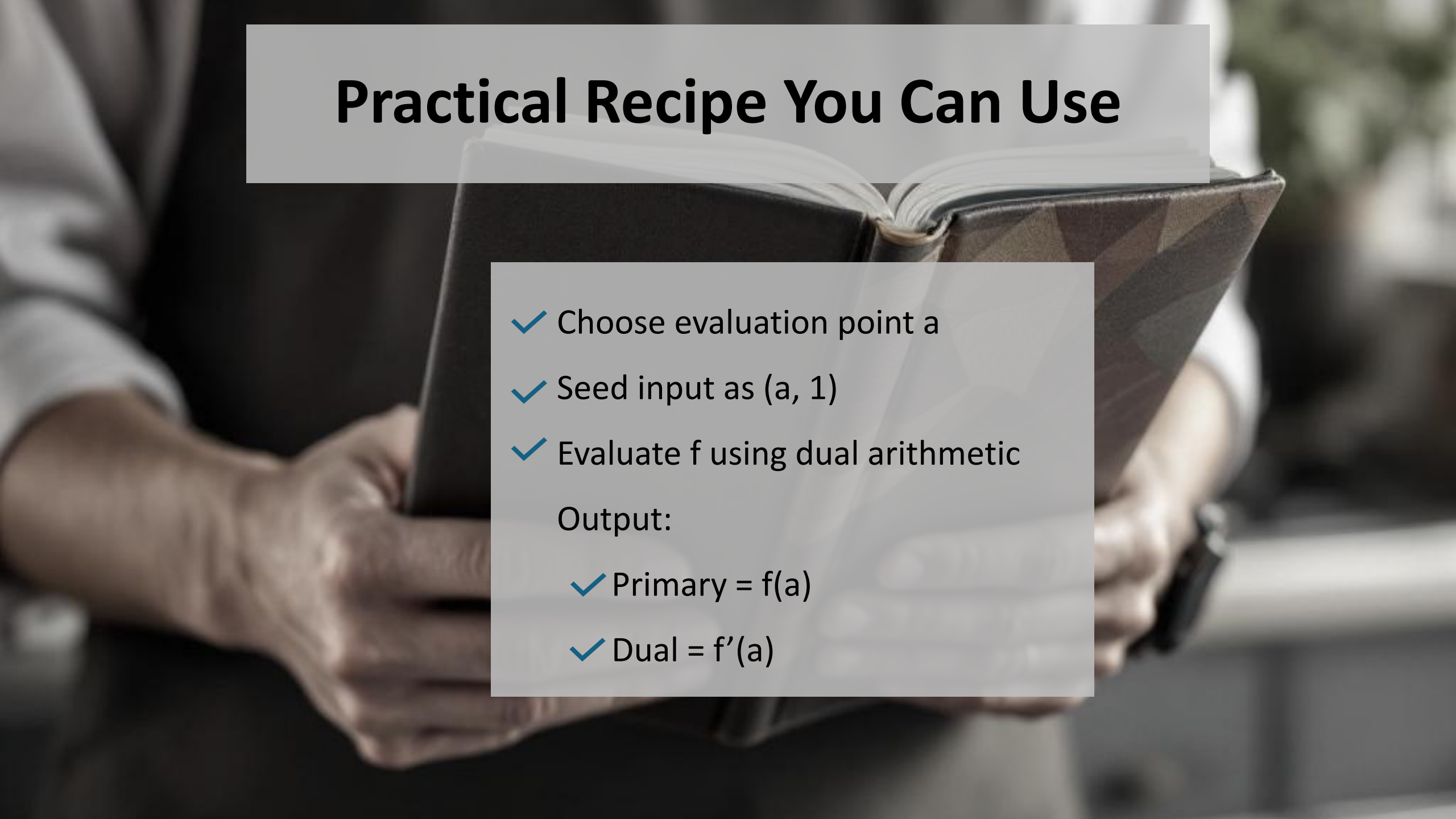
$$\varepsilon \neq 0, \varepsilon^2 = 0$$

- Algebraic device: $\varepsilon^2 = 0$, but $\varepsilon \neq 0$
- Not a real number; lives in an extended algebra
- Captures first-order behavior exactly
- Kills higher-order terms

Unary Functions Library (How ε Propagates)



- Replace $\boldsymbol{f}$ with first-order expansion at $\boldsymbol{a}$
- Coefficient of ε is $\boldsymbol{f'(a) \cdot b}$
- Compose freely: chain rule appears automatically



Practical Recipe You Can Use

- Choose evaluation point a
- Seed input as (a, 1)
- Evaluate f using dual arithmetic

  Output:

  - Primary = f(a)
  - Dual = f’(a)

In-Class Exercise 1 (20 min)

Implement Forward Mode for TinyML Function

Scenario: Sensor data processing for edge deployment

Your Task (Work in pairs):

1. Implement DualNumber class with __sub__, __truediv__, and relu method
2. Compute derivative of scaled with respect to sensor_reading at value 800
3. Verify your answer numerically

- **Time:** 15 minutes coding + 5 minutes presentations
- **Expected Answer:** $\frac{\partial(scaled)}{\partial(sensor_{reading})} = 0.000977$
- Hints: Use python. Import math, numpy

```python
def tinyml_feature_processing(sensor_reading):
# Raw sensor value is 0-1023 (10-bit ADC)
normalized = (sensor_reading - 512) / 512  # Center
around 0
activated = max(0, normalized)  # ReLU activation
scaled = activated * 0.5  # Scale for next layer
return scaled
```

The Processing Pipeline:

Your sensor preprocessing consists of three steps:

1. **Normalize**: Convert raw sensor reading from [0, 1024] to [-1, 1] range
   - Formula: (sensor_value - 512) / 512
   - Why? Neural networks work better with normalized inputs
2. **Activate**: Apply ReLU to remove negative values
   - Formula: max(0, normalized_value)
   - Why? Sometimes we only care about positive deviations
3. **Scale**: Multiply by 0.5 for the final feature
   - Formula: activated_value * 0.5
   - Why? Keeps values in a controlled range for the neural network

Exercise 1 Solution

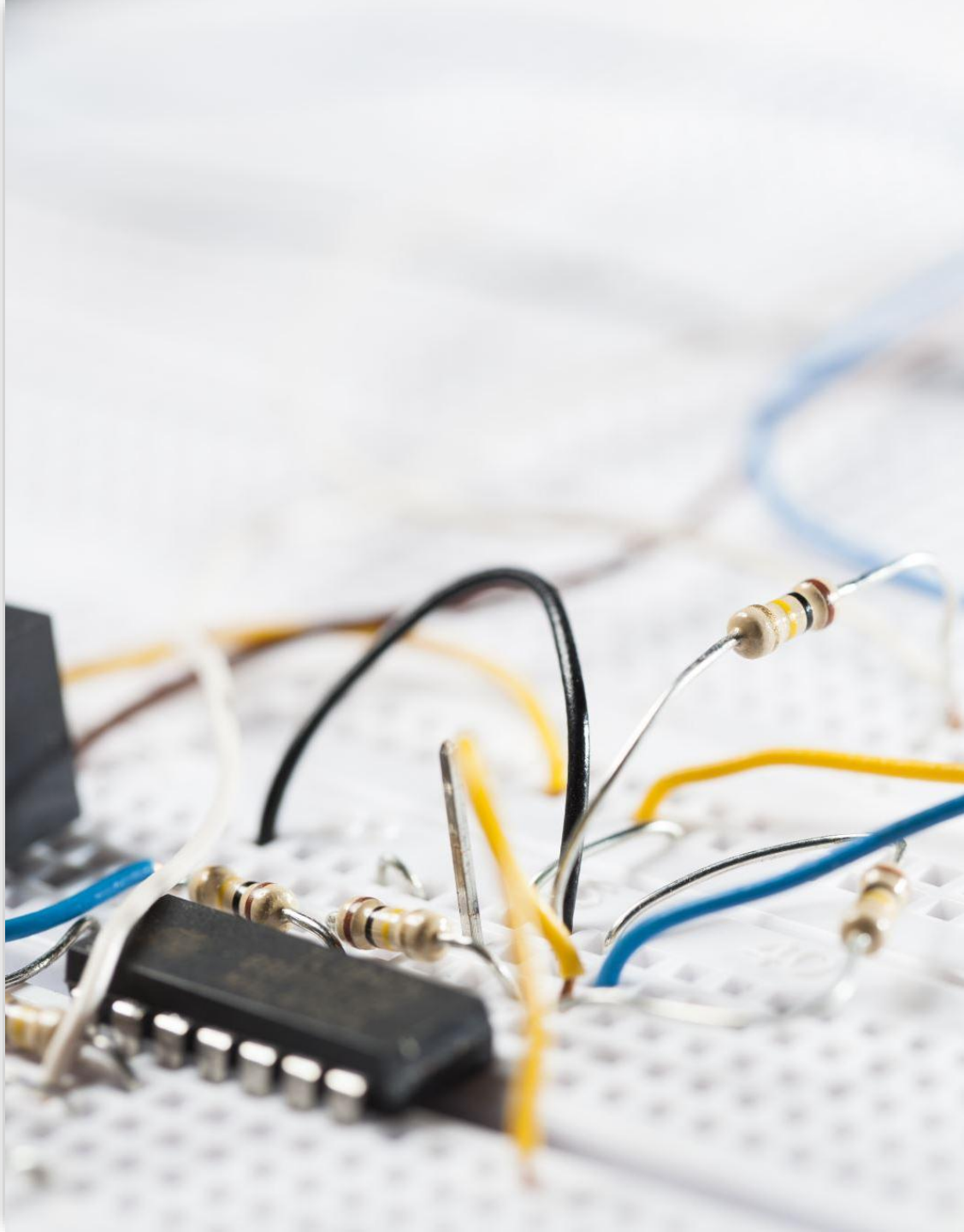
Download the solution (.ipynb - view in JupyterLab)

Forward Mode Performance Analysis

- **Computational Cost**: One derivative computation per original operation
- **Memory Usage**: Constant - stores value + derivative (2x original)
- **Scaling**: Cost = O(n × cost(f)) where n = input variables
- **Trade-off**: Efficiency inversely related to number of input variables

Forward Mode in Neural Networks

- • **Example**: Neural layer with 784 inputs, 128 outputs
  • **Forward Mode Cost**: 784 separate passes × (784×128 operations) = ~78M operations
  • **Memory**: Excellent - constant per input variable
  • **Best Use Cases**:
- Sensitivity analysis
- Feature importance
- Few inputs, many outputs



Forward Mode Applications in TinyML

- **Online Learning**: Single example updates with predictable memory
- **Real-time Sensitivity**: Track sensor importance in real-time
- **Edge Deployment**: Constant resource usage enables deployment
- **Example**: Sensor fusion with importance weighting

```
def analyze_sensor_importance(model, sensor_data):
    # Track how each sensor affects the prediction
    # Forward mode: predictable memory usage
    for sensor_i in range(len(sensor_data)):
    sensitivity = compute_forward_derivative(model,
sensor_data, sensor_i)
    importance_scores[sensor_i] = sensitivity
```

SESSION 2: REVERSE MODE AUTOMATIC DIFFERENTIATION

The Backbone of Neural Network Training

Key Insight: One scalar output (loss), millions of parameters

Two-Phase Process:

- Forward pass: Compute and store intermediates
- Backward pass: Propagate derivatives from output to inputs

Perfect Match: Many inputs, few outputs

Reverse Mode Example - Forward Pass

Function: $f(x) = x^2 \times \sin(x)$
Forward Pass (Store Values):

```
x = 2.0  # Our input value
a = 4.0  # x * x = 2.0 * 2.0
= 4.0
b = 0.909 # sin(2.0) ≈ 0.909
c = 3.637 # a * b = 4.0 *
0.909 ≈ 3.637
```

Key: Store intermediate values and their dependencies
(a and b depend on x, c depends on a and b)

Reverse Mode Example - Backward Pass

Backward Pass

(Compute Gradients):

```
∂c/∂c = 1.0          # Seed the output
∂c/∂a = b = 0.909    # From c = a × b
∂c/∂b = a = 4.0      # From c = a × b

∂c/∂x = ∂c/∂a × 2x + ∂c/∂b × cos(x)
      = 0.909 × 4 + 4.0 × (-0.416)
      = 2.805
```

Chain Rule

Automatically handled
by backward propagation

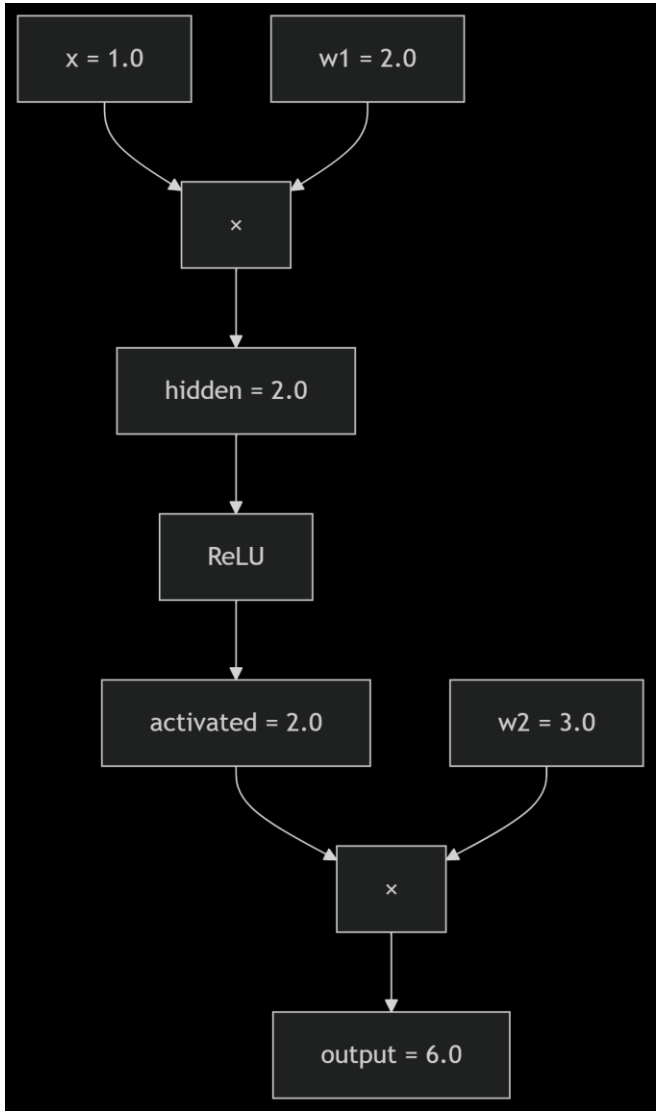
Neural Network Backward Pass

ReLU (Rectified Linear Unit) activation outputs the input directly if it is positive; otherwise, it outputs zero. It's a simple, efficient function that helps introduce non-linearity into neural networks.

- **Simple Network**: `output = ReLU(x × w1) × w2`
- **Forward Values**: `x=1.0, w1=2.0, w2=3.0 → output=6.0`
- **Backward Computation**:

```
∂L/∂output = 1.0                # Start from output
∂L/∂w2 = activated = 2.0         # Use stored forward
value
∂L/∂activated = w2 = 3.0         # Use stored weight
∂L/∂hidden = 3.0 × (1 if hidden>0 else 0) = 3.0
∂L/∂w1 = x × ∂L/∂hidden = 1.0 × 3.0 = 3.0
```

- *__Hidden__ value: the input to the ReLU function*
- ***Activated value**: the output of the ReLU function*



Computational graph for previous slide example

Computational Graph Representation

- **Graph Structure**: Nodes (values) + Edges (operations)
- **Forward**: Compute values, store dependencies
- **Backward**: Follow edges in reverse, accumulate gradients
- **Memory**: Store intermediate values for entire graph
- **Efficiency**: Single pass computes ALL parameter gradients

In-Class Exercise 2 (20 minutes)

Implement Computational Graph for TinyML

- **Scenario**: Wake word detection network (4→3→2 architecture)
- **Task**: Implement computational graph with forward/backward passes
- **Memory Constraint**: Must fit in 1KB total memory
- **Input**: audio_features = [0.2, -0.1, 0.5, 0.3]
- **Target**: [1, 0] (wake word detected)
- **Time**: 15 minutes coding + 5 minutes presentations

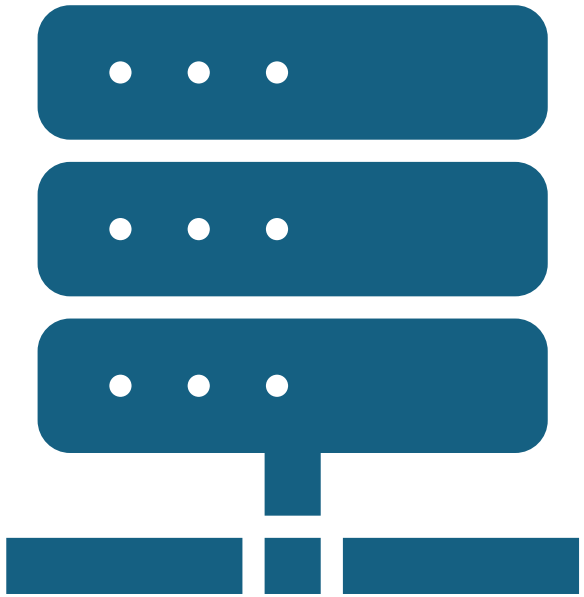
```
def wake_word_classifier(audio_features):  # 4 audio features
    # Hidden layer: 4 -> 3 neurons
    hidden = tanh(W1 @ audio_features + b1)
    # Output layer: 3 -> 2 classes (wake_word, no_wake_word)
    logits = W2 @ hidden + b2
    probs = softmax(logits)
    return probs
```

Exercise 2 Solution

Download the solution (.ipynb - view in JupyterLab)

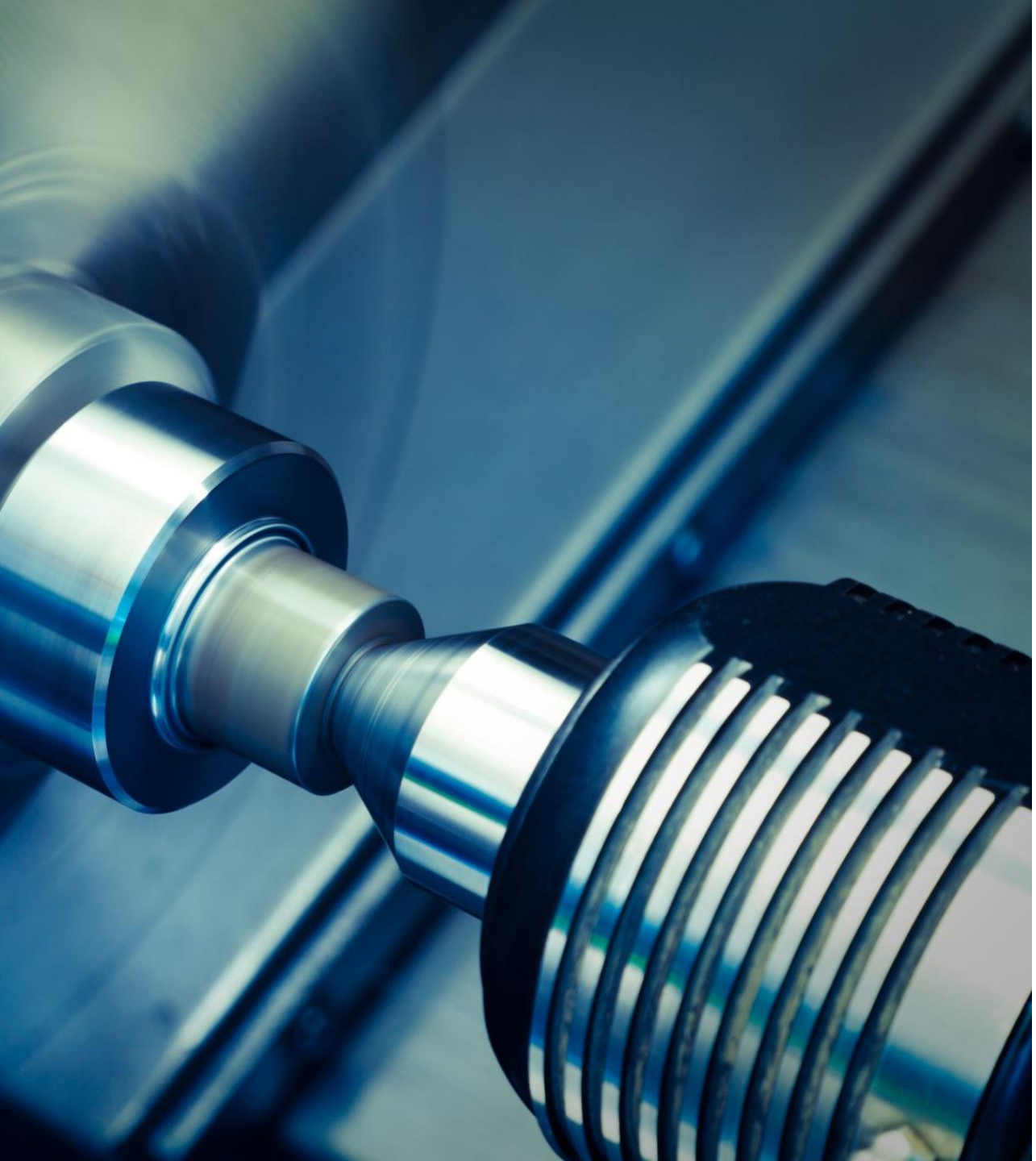
Memory Management - The Critical Challenge

- **The Problem**: Memory grows with network depth
- **Peak Usage**: At start of backward pass (all values stored)
- **Deep Networks**: Gigabytes of intermediate values
- **Challenge**: Balance memory vs computation
- **Impact**: Limits maximum trainable model size



Gradient Checkpointing

- **Strategy**: Store fewer intermediates, recompute during backward
- **Trade-off**: √n memory reduction for ~33% computation increase
- **Example**: Store every k-th layer, recompute the rest
- **Implementation**: Framework manages storage/recomputation automatically
- **Result**: Enables training of much larger models



Advanced Memory Optimizations

- **Operation Fusion**: Combine operations to reduce memory transfers
- **Hardware-Aware Scheduling**: Optimize for specific accelerators
- **Mixed-Precision Training**: 16-bit forward, 32-bit gradients
- **Dynamic Memory Management**: Release memory as soon as possible
- **Result**: Training 100B+ parameter models becomes feasible

```
loss.backward()  # Framework handles everything
```

Framework Integration

User Interface: Simple, intuitive APIs hide complexity

Hidden Complexity:

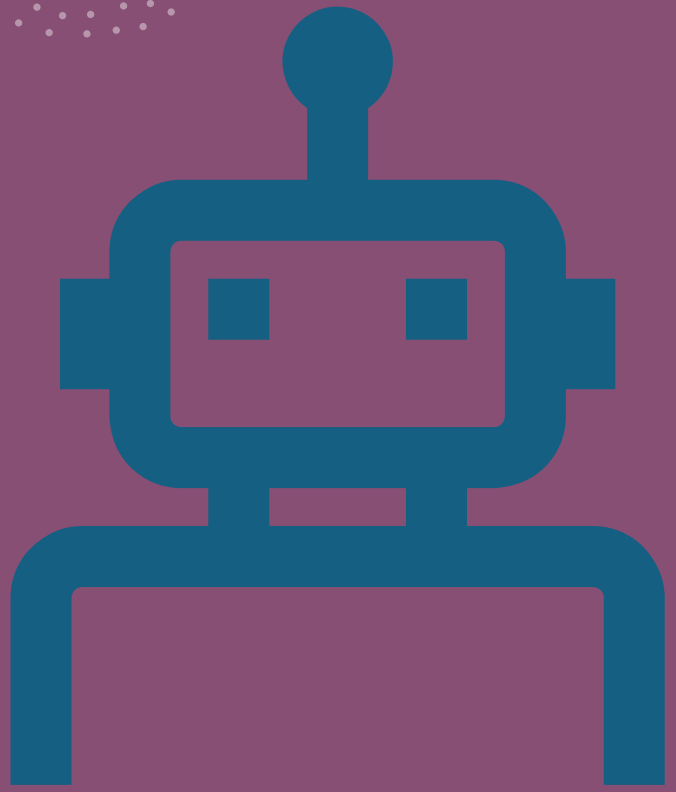
- Graph construction and optimization
- Memory management and scheduling
- Hardware-specific implementations
- Numerical stability considerations

SESSION 3: EMBEDDED SYSTEMS AND PRACTICAL APPLICATIONS

- **Training vs Inference Distinction**
- **Most Embedded**: Inference only (no AD required)
- **Emerging Need**: On-device learning
  - Federated learning
  - Personalization
  - Continual learning
    - **Extreme Constraints**: KB memory, mW power, no OS

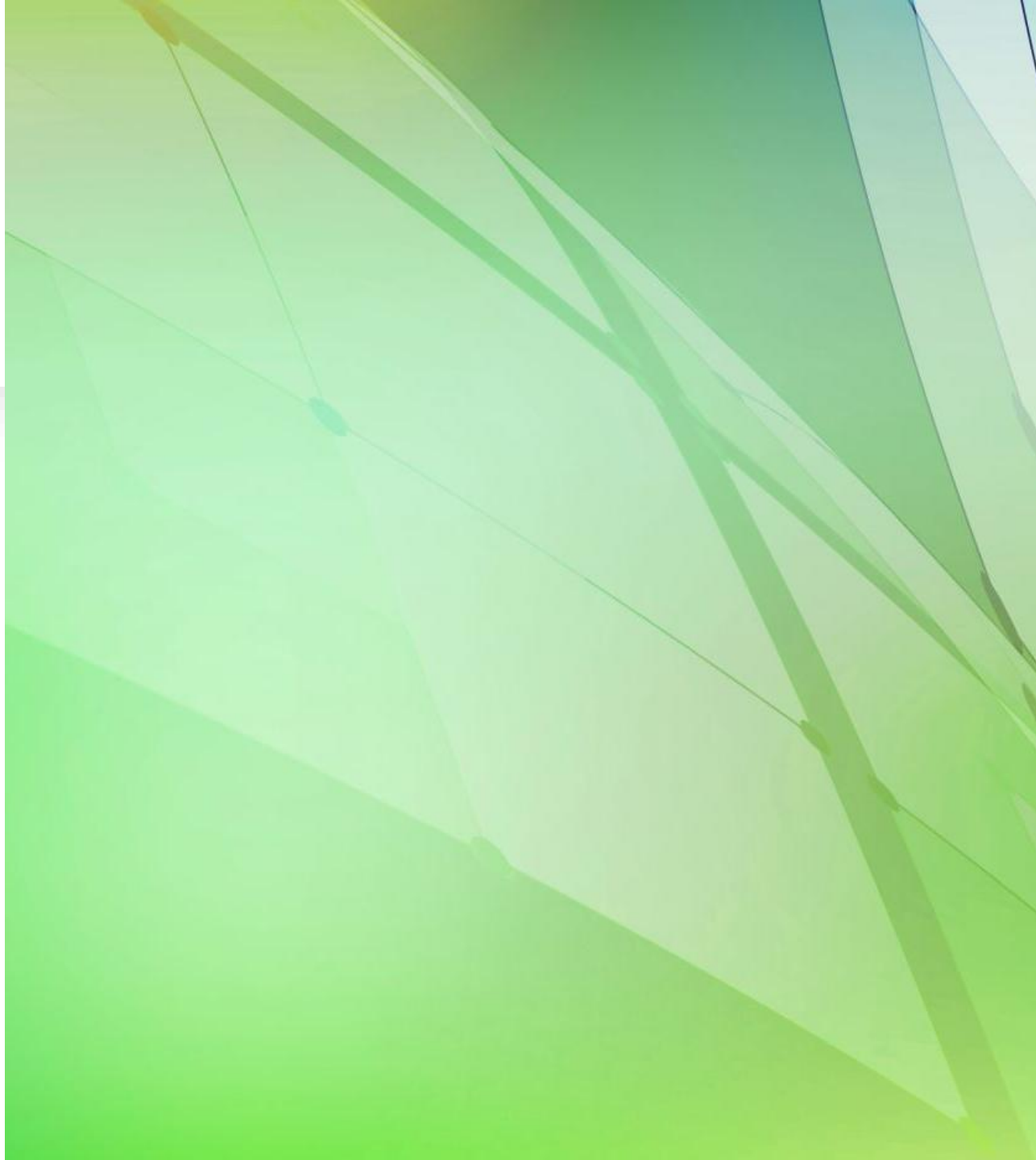
TinyML Gradient Computation

- • **TinyML Characteristics**:
- Energy: < 1 mW
- Memory: Kilobytes total
- No dynamic allocation
- Real-time requirements
  - **Challenge**: Wake word detection with federated learning
  - **Solution**: Aggressive checkpointing with full recomputation



Memory-Efficient AD Implementation

- **Micro-batching**: Process one example at a time
- **Quantized Gradients**: 16-bit or 8-bit gradient computation
- **Static Allocation**: Pre-allocate all memory at compile time
- **Template Optimization**: Generate specialized code for specific architectures
- **Result**: Complete AD system in <5KB binary



Federated Learning Case Study

Scenario: On-device wake word detection

Requirements:

- Local gradient computation on audio
- <32KB total RAM
- Real-time operation
- Federated participation
  - **Implementation**: Forward-only checkpointing + quantization
  - **Results**: 5-10x computation, fits constraints

Comparative Analysis - Forward vs Reverse

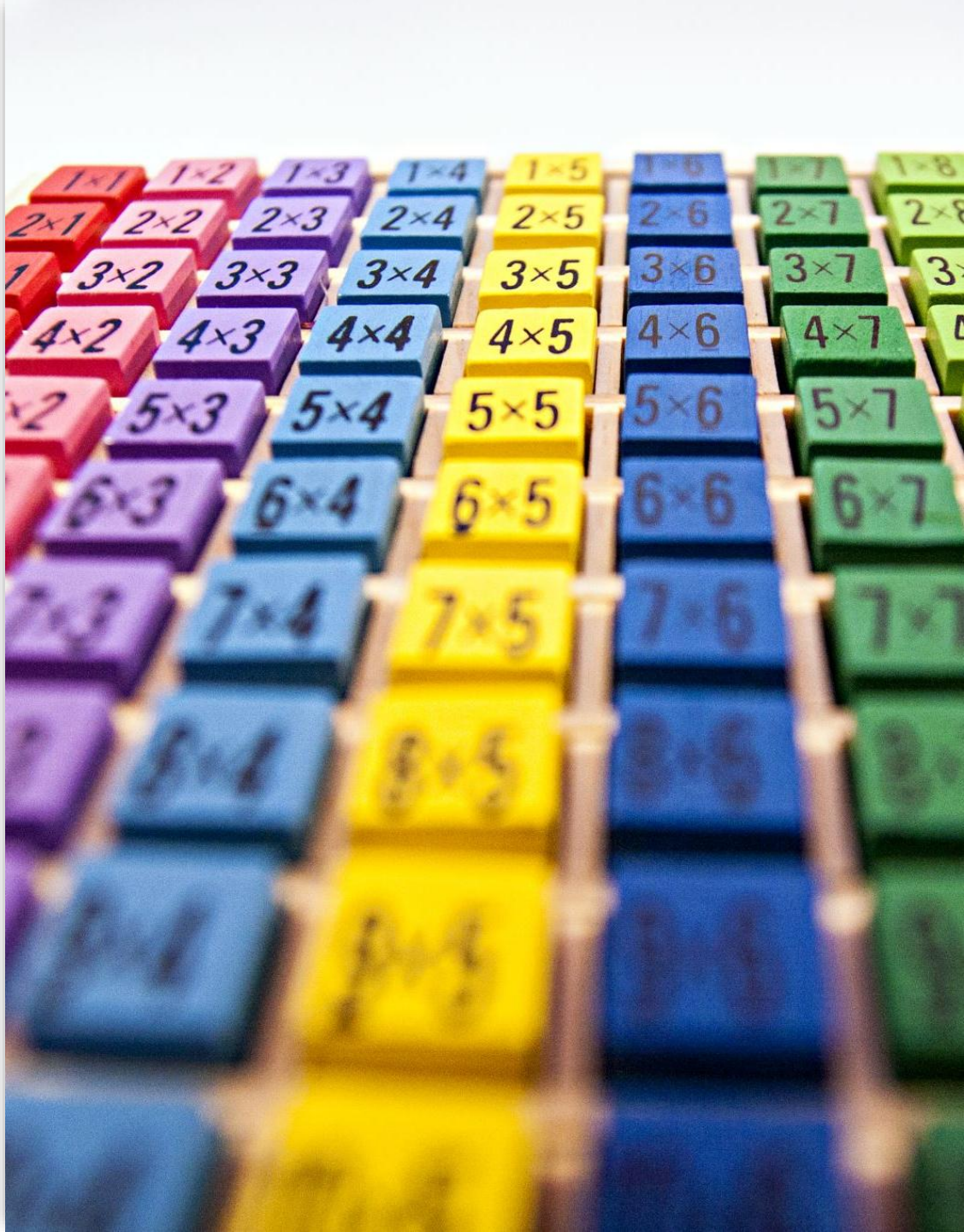
- **ML Training:** Reverse mode dominates (millions of parameters, scalar loss)
- **Analysis Tasks:** Forward mode for sensitivity analysis
- **Embedded Learning:** Hybrid approaches for extreme constraints

System-Level Trade-offs

Aspect	Forward Mode	Reverse Mode
Memory Usage	Constant (2× values)	Linear (store intermediates)
Computation	$O(n \times cost(f))$	$O(m \times cost(f))$
Best For	Few inputs, many outputs	Many inputs, few outputs
Neural Networks	Inefficient for training	Optimal for training
Embedded Systems	Predictable resources	Variable memory needs
Real-time	Suitable	Challenging
Implementation	Operator overloading	Graph construction

Numerical Precision Considerations

- • **Challenge**: Gradients often smaller than activations
  • **Quantization Errors**: Accumulate through chain rule
  • **Mixed-Precision Strategy**:
- Forward: 16-bit activations
- Backward: 32-bit gradients
  • **Benefits**: Reduced memory, maintained stability



In-Class Exercise 3 (20 minutes)

Implement Memory-Efficient AD for Edge Device

Challenge: Gradient computation under extreme memory constraints

Your Task (Groups of 4):

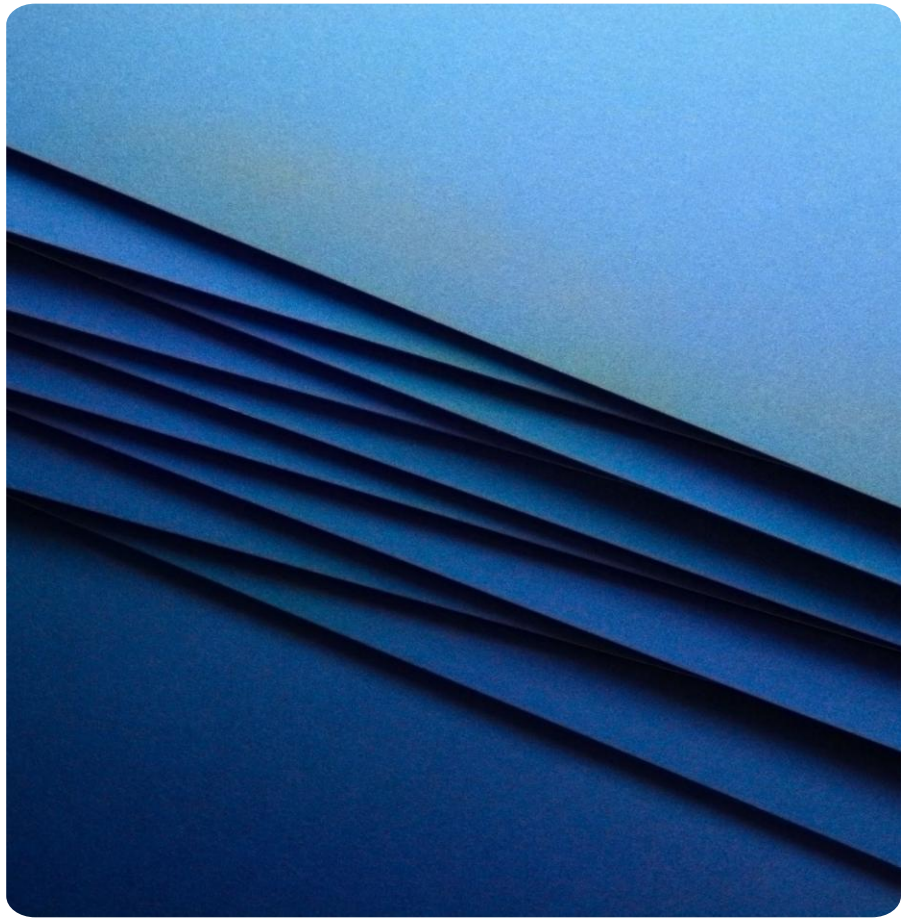
1. **Memory Budget:** 512 bytes total for activations and gradients
2. **Implement:** Checkpointed forward pass with recomputation
3. **Implement:** Quantized gradient computation
4. **Measure:** Actual memory usage and computational overhead
5. **Bonus:** Add federated learning gradient aggregation

```
class EdgeAudioClassifier:
def __init__(self):
    # 3-layer network for audio classification
    self.W1 = np.random.randn(8, 16) * 0.1   # 8
features -> 16 hidden
    self.b1 = np.zeros(16)
    self.W2 = np.random.randn(16, 8) * 0.1   # 16
-> 8 hidden
    self.b2 = np.zeros(8)
    self.W3 = np.random.randn(8, 3) * 0.1 # 8 -> 3
classes
    self.b3 = np.zeros(3)

def forward(self, x):
    z1 = self.W1.T @ x + self.b1
    a1 = np.maximum(0, z1)  # ReLU
    z2 = self.W2.T @ a1 + self.b2
    a2 = np.maximum(0, z2)  # ReLU
    z3 = self.W3.T @ a2 + self.b3
    return softmax(z3)
```

Exercise 3 Solution

Download the solution (.ipynb - view in JupyterLab)



Future Directions

Hardware Acceleration:

AD-specific processors and memory hierarchies

Advanced Techniques:

- Differentiable programming
- Probabilistic AD
- Approximate gradients
  - **Research Opportunities**: Novel memory strategies, hybrid approaches, domain-specific optimizations

Summary - Week 2 Key Takeaways

- **Algorithmic Insights**: Forward vs reverse mode trade-offs
- **System Design**: Memory management is critical for scalability
- **Implementation**: Different strategies for different constraints
- **Practical Impact**: AD enables modern ML, but implementation matters
- **Key Lesson**: Understanding trade-offs guides optimal deployment

Looking Ahead - Week 3 Preview

Graph-Level Optimizations

Next Week's Focus:

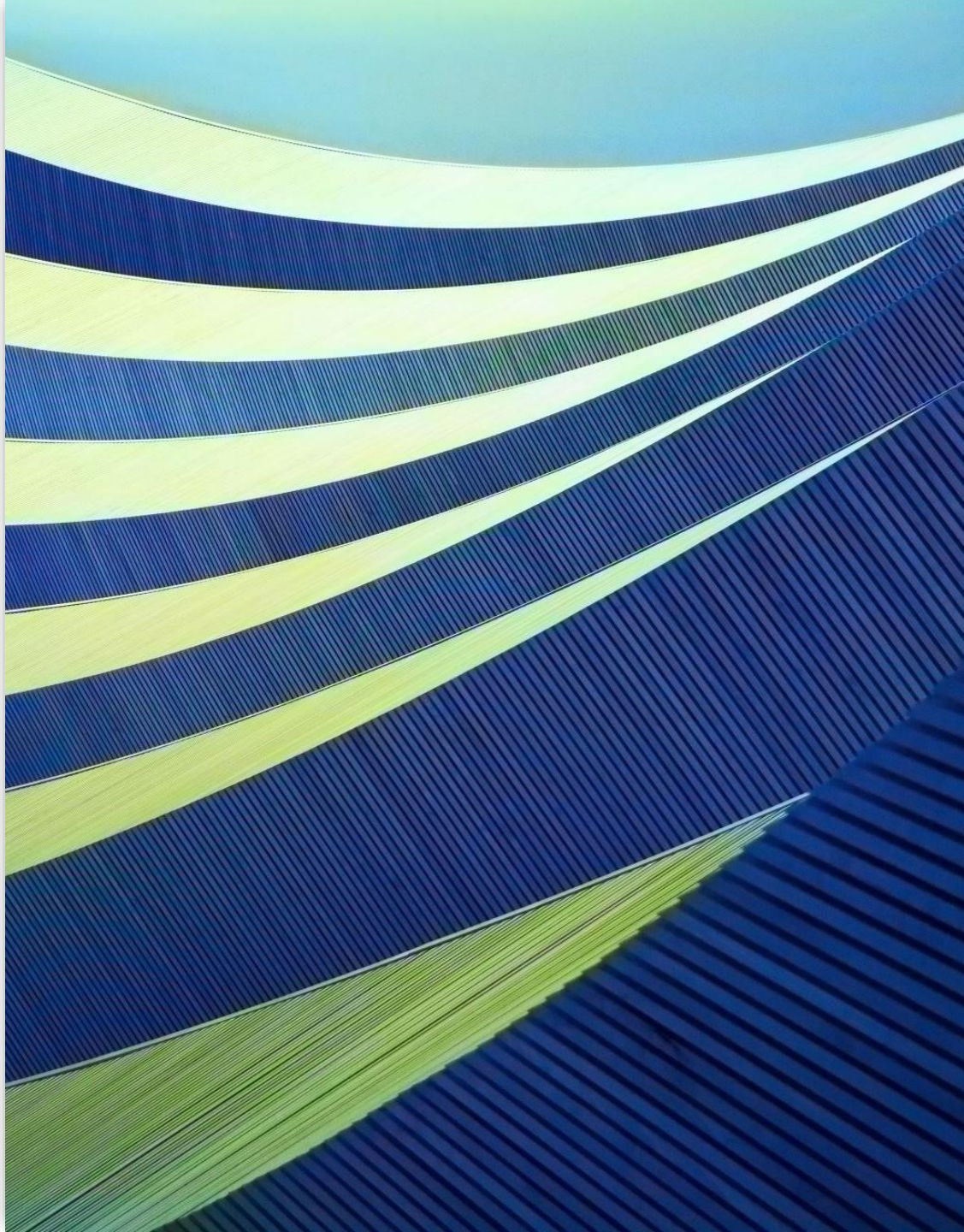
- Computational graph analysis and transformation
- Operation fusion and kernel optimization
- Memory access pattern optimization
- Compilation strategies for ML workloads

Connection to AD:

- Graph optimizations must preserve gradient computation
- Fusion strategies affect backward pass implementation
- Memory optimizations impact AD memory management

Required Readings for Week 3:

- Book 1 – Volume 1 : ML Frameworks
- Book 1 – Volume 1 : Hardware Acceleration
- Book 1 – Volume 1 : Model Serving
- Book 2: Chapter 19: Porting Models from TensorFlow to TensorFlow Lite
- Book 2: Chapter 15: Optimizing Latency



Questions and Discussion

Week 2 Wrap-up

Today's Key Concepts:

- Forward vs reverse mode automatic differentiation
- Memory and computational trade-offs in AD systems
- Implementation strategies for resource-constrained environments
- System-level considerations for embedded deployment

Discussion Questions:

- When would you choose forward mode over reverse mode?
- How do memory constraints affect gradient computation strategy?
- What are the implications of quantized gradients for training stability?